

FPGA Station

1 0 1 1 0 1 0 1 1 1 0 0 1 1 0 1 0 1 0 1 0 1
0 1 1 0 1 1 0 0 0 1 1 1 0 1 0 1 0 0 1 1 0 1 1 1 0
1 1 1 0 0 0 1 1 0 1 0

Hardware Accelerators and much more

HARDWARE IMPLEMENTATION OF REGRESSION TSETLIN MACHINE

Internship Report
of

Ashwin Tojo
FS2026003

Indian Institute of Technology, Bhubaneswar
School of Electrical and Computer Sciences
Dept. of Electronics & Communication Engineering

Supervisor: Prof. Dr.Ing Srinivas Boppu

Mentor: K Koushik

July 20, 2026

Abstract

This internship presents the hardware implementation of a Regression Tsetlin Machine (RTM) on the Xilinx ZCU104 FPGA platform. The project began with the software implementation of the RTM using the California Housing dataset in Google Colaboratory. The trained model generated the required memory files, including clause, weight, and test data files, which were subsequently used for hardware implementation.

The hardware design was developed using Verilog HDL in Xilinx Vivado. The implementation includes the Clause Matching Function, memory initialization, and prediction logic required to perform regression on FPGA hardware. The design was synthesized, implemented, and integrated with the Processing System (PS) of the Zynq UltraScale+ MPSoC. The generated hardware platform was exported to Xilinx Vitis, where the application was developed and deployed onto the ZCU104 evaluation board.

The hardware implementation was validated using the generated test data, and the prediction accuracy was evaluated using the Mean Absolute Error (MAE). The experimental results demonstrate that the FPGA implementation successfully reproduces the functionality of the software model while providing an efficient hardware realization of the Regression Tsetlin Machine. This work highlights the feasibility of implementing interpretable machine learning algorithms on FPGA platforms and provides practical experience in FPGA design, hardware–software co-design, and machine learning acceleration.

Contents

1	Introduction	3
1.1	Regression	3
1.2	Tsetlin Machine	3
1.3	Regression Tsetlin Machine	3
2	Software Implementation	4
2.1	Overview	4
2.2	Regression Tsetlin Machine Workflow	4
2.3	California Housing Dataset	4
2.4	Clause Bank and Weight Bank	4
2.5	Generated Memory Files	5
2.6	Performance Evaluation	5
3	Hardware Implementation	6
3.1	Overview	6
3.2	Clause Matching Function	6
3.3	Testbench Implementation	6
3.4	Fixed-Point Representation	7
3.5	Prediction Computation	7
3.6	Mean Absolute Error Calculation	7
3.7	Datapath and Controller Architecture	8
3.8	Hardware Modules	8
3.8.1	Clause Matching Function	8
3.8.2	Accumulator	8
3.8.3	Controller	9
3.8.4	Counters	9
3.8.5	MeanAbsolute Error (MAE) Calculator	9
3.8.6	Top Module	9
3.8.7	Testbench	10
3.8.8	Important Note	10
3.9	Block RAM Implementation	10
3.10	Hardware Modifications for Block RAM Integration	11
3.11	Modified Controller	12
3.12	Block Design	12
4	Using Vitis Platform	13
4.1	Software Workflow in Vitis	13
4.2	Running the Design on Hardware	14

5	Experimental Setup	15
5.1	Hardware Setup	15
5.2	Software Setup	16
6	Results	17
6.1	Software Results	17
6.2	Hardware Results in VIVADO with or without BRAM	17
6.3	Hardware Results in VITIS	17
6.4	Performance Comparison	18
7	Conclusion	19
	References	19

1 Introduction

1.1 Regression

Regression is a supervised machine learning technique used to predict continuous numerical values based on one or more input features. Unlike classification, which predicts discrete categories, regression estimates real-valued outputs. It is widely used in applications such as house price prediction, stock market analysis, weather forecasting, and demand estimation. The objective of a regression model is to learn the relationship between input variables and the target variable while minimizing the prediction error.

1.2 Tsetlin Machine

The Tsetlin Machine is a machine learning algorithm based on propositional logic and Tsetlin Automata. It learns by constructing logical expressions, known as clauses, that capture patterns in the input data. Compared to many conventional machine learning models, the Tsetlin Machine provides high interpretability, low computational complexity, and competitive accuracy. These characteristics make it suitable for both software and hardware implementations.

1.3 Regression Tsetlin Machine

The Regression Tsetlin Machine (RTM) is an extension of the Tsetlin Machine designed for regression tasks involving continuous-valued outputs. Instead of predicting class labels, the RTM estimates numerical values by combining the outputs of multiple logical clauses. It preserves the interpretability of the original Tsetlin Machine while enabling regression capabilities. In this project, the Regression Tsetlin Machine is implemented in hardware on an FPGA to achieve efficient and accelerated computation.

for more information on RTM refer <https://royalsocietypublishing.org/rsta/article/378/2164/20190165/111541/The-regression-Tsetlin-machine-a-novel-approach-to>

2 Software Implementation

2.1 Overview

The software implementation was carried out using Google Colaboratory. The Regression Tsetlin Machine (RTM) code for the California Housing dataset was first obtained and executed in Google Colab. The program was divided into multiple code cells, and each cell was executed sequentially to complete the training and testing process. Upon successful execution, the trained model generated the memory files required for the hardware implementation.

2.2 Regression Tsetlin Machine Workflow

The workflow begins by loading the California Housing dataset, which consists of input features (X) and corresponding target values (Y). The dataset is divided into training and testing sets, namely X_{train} , X_{test} , Y_{train} , and Y_{test} .

During training, the Regression Tsetlin Machine learns a set of logical clauses together with their corresponding weights. These learned parameters are stored in the Clause Bank and Weight Bank, respectively. After training is completed, the learned parameters are exported as memory files, which are later used for FPGA implementation.

2.3 California Housing Dataset

The California Housing dataset is used to train and evaluate the Regression Tsetlin Machine. After preprocessing, each input sample is represented using 80 binary features. The dataset consists of two components:

- X – Input feature vectors.
- Y – Corresponding target values.

The dataset is divided into training and testing sets to evaluate the prediction performance of the trained model.

2.4 Clause Bank and Weight Bank

The Clause Bank stores the logical clauses learned during training. Each clause represents a logical rule formed using literals and their negated counterparts. Since there are 80 literals and their corresponding 80 negated literals, every clause occupies a total of 160 bits.

The Weight Bank stores the weights associated with each clause. During prediction, these weights determine the contribution of each clause toward the final regression output.

2.5 Generated Memory Files

The execution of the software implementation generates four memory files that are required for hardware implementation.

Memory File	Bit Width	Number of Entries
clauses.mem	160 bits	1000
weights.mem	32 bits	1000
Xtest.mem	160 bits	5160
Ytest.mem	32 bits	100

Table 1: Memory files generated by the software implementation.

The `clauses.mem` file stores the learned logical clauses, while `weights.mem` stores the corresponding clause weights. The `Xtest.mem` file contains the test feature vectors used during hardware testing, whereas `Ytest.mem` contains the expected output values for performance evaluation.

link for the .mem files -> https://github.com/koushikkalyani/FPGA-station/tree/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/MEM_FILES.

2.6 Performance Evaluation

The prediction accuracy of the Regression Tsetlin Machine is evaluated using the Mean Absolute Error (MAE). During testing, each input vector from `Xtest.mem` is processed to obtain a predicted output. The predicted value is then compared with the corresponding value in `Ytest.mem`. The absolute difference between these values is calculated for every test sample, and the average of all absolute differences gives the Mean Absolute Error.

The MAE is calculated using

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

where y_i represents the actual output, $\hat{y}_i$ represents the predicted output, and N is the total number of test samples. A lower MAE indicates better prediction accuracy.

Python Code for Software Implementation -> [https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/SOFTWARE/California_Housing_Python\(RTM\).ipynb](https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/SOFTWARE/California_Housing_Python(RTM).ipynb)

3 Hardware Implementation

https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/reference_code/Code_using_for_loops. The above code is provided solely for understanding the implementation and was not used in the final hardware implementation. And its explanation is given below till page 7.

3.1 Overview

The memory files generated during the software implementation, namely `clauses.mem`, `weights.mem`, `Xtest.mem`, and `Ytest.mem`, are first downloaded from the Google Colaboratory environment. These files serve as the input data for the hardware implementation and are imported into the Vivado project for simulation and verification.

The hardware implementation focuses on reproducing the inference stage of the Regression Tsetlin Machine. The objective is to evaluate each input sample using the learned clauses and weights and compare the predicted output with the expected output stored in `Ytest.mem`.

3.2 Clause Matching Function

The Clause Matching Function forms the fundamental building block of the hardware implementation. Every input sample is compared with each learned clause to determine whether the clause is satisfied.

The comparison is performed using simple bitwise logic. Each clause consists of 160 bits, corresponding to 80 literals and their 80 negated literals. A bitwise OR operation between the complemented clause and the input vector is performed, followed by a reduction AND operation. If every bit satisfies the logical condition, the clause is considered matched and produces an output of logic '1'. Otherwise, the output is '0'.

Since only bitwise operations are required, the clause matching function is highly suitable for FPGA implementation.

3.3 Testbench Implementation

A Verilog testbench is developed to verify the functionality of the hardware implementation. The testbench reads the memory files generated during software execution using the `readmemb` system task. The four memory files contain the learned clauses, clause weights, test input vectors, and expected output values.

The hardware evaluates the first 100 test samples. For every input sample, all 1000 learned clauses are processed sequentially. Whenever a clause is satisfied, its corresponding weight is added to an accumulator. The accumulated value represents the raw output of the Regression Tsetlin Machine before scaling.

3.4 Fixed-Point Representation

The original software implementation produces regression outputs in the range of approximately 0.14999 to 5.00001. Since floating-point arithmetic requires significant hardware resources, it is avoided in the FPGA implementation.

Instead, all regression values are multiplied by 100000 before being stored in the memory files. Consequently, the range

$$0.14999 \leq y \leq 5.00001$$

is represented as

$$14999 \leq y \leq 500001.$$

This fixed-point representation enables all computations to be performed using integer arithmetic, thereby simplifying the hardware design while preserving numerical precision.

3.5 Prediction Computation

After obtaining the accumulated clause weight, the predicted regression value is computed using the scaling equation employed by the Regression Tsetlin Machine. The prediction is transformed back into the original output range using the predefined maximum value, minimum value, and threshold parameters.

The predicted output is then compared with the corresponding reference value stored in Ytest.mem.

3.6 Mean Absolute Error Calculation

The performance of the hardware implementation is evaluated using the Mean Absolute Error (MAE). For every test sample, the absolute difference between the predicted output and the expected output is calculated.

The absolute errors of all 100 test samples are accumulated, and the average error is computed in the same manner as before with N=100.

Since the regression values are stored after multiplication by 100000, the accumulated error is divided by 100000 before reporting the final MAE value, ensuring that the reported error corresponds to the original regression scale.

Intermediate variables such as the accumulated clause weights, multiplication results, and error values are declared as signed 64-bit registers (reg signed [63:0]). The wider bit width prevents arithmetic overflow during multiplication, accumulation, and MAE computation, ensuring accurate fixed-point calculations throughout the hardware implementation.

3.7 Datapath and Controller Architecture

The initial hardware implementation was developed using nested for loops to iterate over the clauses and test samples. While this approach is suitable for simulation, it cannot be directly synthesized into efficient hardware. A for loop in Verilog does not represent a hardware component; instead, it is an abstraction used by the simulator or synthesis tool. Implementing large nested loops would require excessive hardware resources and would not provide a sequential execution mechanism suitable for FPGA implementation.

To overcome this limitation, the iterative operations are replaced by dedicated hardware counters that generate the required addresses for clauses and test samples. The remaining functionality is divided into independent hardware modules, each performing a specific task. These modules communicate through control signals generated by a finite state machine (FSM), resulting in a datapath-controller architecture. This modular approach improves readability, simplifies debugging, and closely follows standard digital design practices.

3.8 Hardware Modules

The complete hardware implementation is divided into several functional modules, each responsible for a specific operation during the inference process.

3.8.1 Clause Matching Function

The Clause Matching Function compares the current input sample with a learned clause and determines whether the clause is satisfied. The comparison is performed using combinational logic consisting of bitwise operations followed by a reduction AND operation. A logic high output indicates that the clause is satisfied and should contribute to the final prediction.

for code -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/MODULES/Clause_Matching_function.v

3.8.2 Accumulator

The Accumulator stores the running sum of the clause weights during inference. Whenever a clause is satisfied, the corresponding weight is added to the accumulated value. At the beginning of every new test sample, the accumulator is reset to zero before processing the next input vector.

for code -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/MODULES/Accumulator_california.v

3.8.3 Controller

The Controller is implemented as a Finite State Machine (FSM) that coordinates the complete inference process. It generates the control signals required to load the memory contents, evaluate clause matching, update the accumulator, calculate the predicted output, and process successive test samples. The controller also determines when the complete evaluation has finished and asserts the completion signal.

for code -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/MODULES/CONTROLLER.v

The above code contains both codes for BRAM implementation and without BRAM implementation use either one as per need. Currently we are in without BRAM section.

3.8.4 Counters

Two hardware counters replace the nested software loops used during simulation. One counter sequentially accesses the learned clauses, while the second counter iterates through the test samples. These counters generate the required memory addresses and indicate when all clauses or all samples have been processed.

for code -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/MODULES/COUNTER_california.v

3.8.5 MeanAbsolute Error (MAE) Calculator

The MAE Calculator converts the accumulated clause score into the corresponding regression output using the scaling equation employed by the Regression Tsetlin Machine. The predicted output is compared with the expected value stored in `Ytest.mem`, and the absolute error is calculated. The absolute errors of all test samples are accumulated to obtain the final Mean Absolute Error.

Since the hardware implementation uses fixed-point arithmetic, all regression values are represented as scaled integers. Intermediate calculations involve multiplication and accumulation of large numerical values, which may exceed the range of a 32-bit register. Therefore, 64-bit signed registers are used for intermediate variables to prevent arithmetic overflow and preserve computational accuracy throughout the MAE calculation.

for code -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/MODULES/MAE_Calculator.v

3.8.6 Top Module

The Top Module integrates all individual hardware modules into a single design. It instantiates the Clause Matching Function, Accumulator, Controller, Counters, and MAE

Calculator, while also interfacing with the memory files containing the learned clauses, clause weights, test inputs, and expected outputs. It represents the complete hardware realization of the Regression Tsetlin Machine inference engine.

for code -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/MODULES/TOPMODULE.v

The above code contains both codes for BRAM implementation and without BRAM implementation use either one as per need. Currently we are in without BRAM section.

3.8.7 Testbench

A dedicated Verilog testbench is developed to verify the functionality of the complete hardware design. The testbench generates the system clock and reset signals, initiates the inference process, waits for the completion signal from the controller, and finally displays the computed Mean Absolute Error. This verifies that the hardware implementation produces results consistent with the software implementation.

for code -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/MODULES/TESTBENCH.v

3.8.8 Important Note

Do remember to press runall in vivado because the time given in testbench is not enough for the whole simulation to complete.

3.9 Block RAM Implementation

The memory files generated during software execution are initially available in the .mem format. Although these files are suitable for simulation using the readmemb system task, they cannot be directly initialized into the Block RAM (BRAM) resources of the FPGA. Vivado Block Memory Generator requires initialization files in the .coe (Coefficient) format. Therefore, all four memory files, namely clauses.mem, weights.mem, Xtest.mem, and Ytest.mem, are converted into their corresponding .coe files before hardware implementation.

link for conversion of .mem file to .coe file -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/SOFTWARE/mem_to_coe_conversion.ipynb

link for the .coe files -> https://github.com/koushikkalyani/FPGA-station/tree/main/Project-depot/FS2026003/HARDWARE/VERILOG_HARDWARE/COE_FILES.

Steps to implement BRAM are given below:

1. Open the Vivado project.

2. In the **IP Catalog**, search for **Block Memory Generator** and add the IP to the project.
3. Open the Block Memory Generator customization window.
4. Configure the memory width and depth according to the corresponding memory file.
5. Under the **Other Options** tab, select **Load Init File**.
6. Browse and select the corresponding `.coe` file.
7. Click **OK** and generate the output products for the IP.
8. Repeat the same procedure for `clauses.coe`, `weights.coe`, `Xtest.coe`, and `Ytest.coe`.
9. Instantiate all four Block RAM IPs in the top-level hardware design.
10. Connect the address, clock, enable, and data output ports of each Block RAM to the datapath and controller.
11. Ensure the clock latency is 1 clock cycle and not 2, as our code is written wrt to it.

Four independent Block RAMs are created:

Clause BRAM Weight BRAM Xtest BRAM Ytest BRAM

Each BRAM stores one of the generated memory files and provides synchronous read access during inference.

3.10 Hardware Modifications for Block RAM Integration

The original hardware implementation accessed memory using Verilog arrays together with the `readmemb` system task. While this approach is sufficient for simulation, it cannot be synthesized into dedicated FPGA memory resources. Therefore, these memory arrays are replaced by Block RAM IP cores generated using Vivado.

Instead of directly indexing Verilog arrays, the controller now supplies the memory addresses to the corresponding Block RAMs. Each Block RAM returns the requested data through its output port, which is subsequently processed by the remaining hardware modules.

Unlike Verilog arrays used during simulation, Block RAMs operate synchronously and introduce a one-clock-cycle read latency. Consequently, valid data does not become available immediately after an address is applied. To accommodate this latency, additional controller states are inserted into the finite state machine. These intermediate states allow sufficient time for the requested data to be read from the Block RAM before being consumed by the datapath.

The controller therefore follows the sequence:

Apply memory address. Wait for one clock cycle. Read valid data from Block RAM. Perform clause evaluation. Continue with the next operation.

Introducing these additional wait states ensures proper synchronization between the controller and the Block RAMs, preventing incorrect memory accesses and guaranteeing correct hardware operation.

3.11 Modified Controller

The controller was modified to include additional states corresponding to Block RAM read latency. Separate load states were introduced before clause evaluation and output computation to ensure that valid memory data was available before processing. Apart from these synchronization states, the overall inference sequence remained unchanged.

3.12 Block Design

Step to Create Block Design

Step 1: In Vivado, select Create Block Design. Add the Zynq UltraScale+ MPSoC IP and run Block Automation to configure the PS for the ZCU102 board preset. Add the tmwrapper module as a custom RTL IP. Add three AXI GPIO blocks: GPIO 0: Start signal (output from PS to PL). GPIO 1: Done signal (input from PL to PS). GPIO 2: Correct count (input from PL to PS). Run Connection Automation to connect the AXI interfaces via the AXI SmartConnect. Connect the GPIO signals to the tmwrapper ports. Validate the block design.

Step 2: Generate Output Products and HDL Wrapper Right-click the block design and select Generate Output Products (Global synthesis). Right-click again and select Create HDL Wrapper (Let Vivado manage wrapper and auto-update).

Step 3: Synthesis, Implementation, and Bitstream Generation Click Run Synthesis and wait for completion. Click Run Implementation and wait for completion. Review timing, utilization, and power reports. Click Generate Bitstream to produce the FPGA programming file.

Step 4: Export Hardware (.xsa). Go to File → Export → Export Hardware. elect Include Bitstream to bundle the bitstream with the hardware handoff. Save the .xsa file.

link for .xsa file -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VITIS_RELATED_FILE/BLOCK_design_California_wrapper_2025_newest.xsa

link for Block Design -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/BLOCK_DESIGN/BLOCK_design_California.bd

4 Using Vitis Platform

4.1 Software Workflow in Vitis

1. Launch Vitis

- Open AMD Xilinx Vitis 2025.1.
- Select or create a workspace.

2. Create the Platform Project

- Select File → New → Platform Component.
- Provide a platform name.
- Select the exported XSA file generated from Vivado.
- Allow Vitis to generate the platform.
- Ensure that the FSBL and PMU Firmware are generated.

3. Build the Platform

- Right-click the Platform Component.
- Select Build Project.
- Verify that the platform builds successfully.

4. Create the Application Project

- Select File → New → Application Component.
- Choose the previously created platform.
- Provide the application name.
- Select the Standalone domain.
- Choose the target processor (Cortex-A53 #0).
- Select the required application template.

5. Add Source Files

- Replace the default `main.c`

link for the main.c file -> https://github.com/koushikkalyani/FPGA-station/blob/main/Project-depot/FS2026003/HARDWARE/VITIS_RELATED_FILE/main.c
- Add all application source (.c) and header (.h) files.
- Save the project.

6. Build the Application

- Right-click the Application Component.
- Select Build Project.
- Verify that the executable (.elf) file is generated successfully.

4.2 Running the Design on Hardware

1. Power on the ZCU104 Board

- Connect the JTAG and USB-UART cables.
- Power on the ZCU104 board.

2. Launch the Hardware Server

- Open terminal and type hw_server
- Ensure that the hw_server is running.

3. Open the Serial Monitor

- Open fresh terminal and type "ls /dev/ttyUSB*" decide the correct port
- Open the Serial Monitor in Vitis.
- Select the correct UART port.
- Set the baud rate to 115200.

4. Verify the Processor Status

- Open fresh terminal and type xsct.
- Execute one by one :

connect
targets
- Verify that the ZCU104 board is detected.

5. Program the FPGA

- Click Program Device.
- Wait until programming completes successfully.
- Verify that Cortex-A53 #0 changes to the **Running** state.

6. Run the Application

- Select the Application Component.

- Click Run.

7. Observe the Output

- Monitor the Serial Monitor window.
- The expected output is:

```
California FPGA Start  
Sending start pulse  
Waiting for done...  
Done!  
MAE = 44237
```

5 Experimental Setup

5.1 Hardware Setup

The hardware experiments were performed using the following equipment:

- Xilinx ZCU104 Evaluation Board

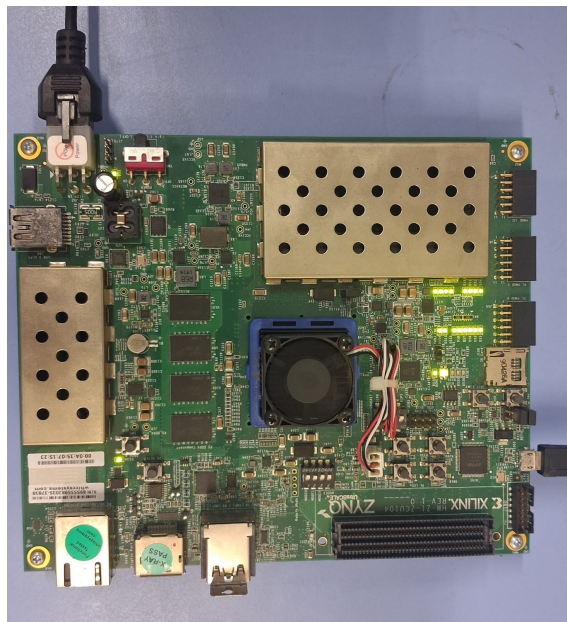


Figure 1: Xilinx ZCU104 Evaluation Board

- 12 V DC Power Adapter



Figure 2: Power Adapter

- Micro-USB cable (USB-JTAG/USB-UART interface)



Figure 3: Micro-USB cable

- Host computer running Ubuntu Linux

5.2 Software Setup

The software implementation and hardware development were carried out using the following tools:

- Google Colaboratory – Training the Regression Tsetlin Machine and generating the memory files.
- Python – Executing the RTM software model.
- Xilinx Vivado 2025.1 – Hardware design, synthesis, implementation, and bitstream generation.
- Xilinx Vitis 2025.1 – Platform creation, application development, hardware deployment and for observing the output.

6 Results

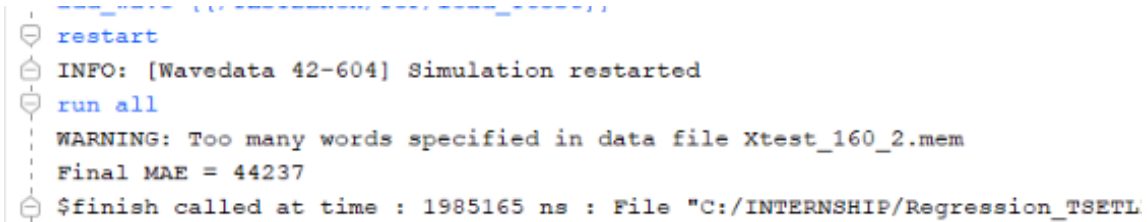
6.1 Software Results



```
FINAL RESULTS
FINAL MAE: 0.4469779644345881
```

Figure 4: MAE from PYTHON implementation

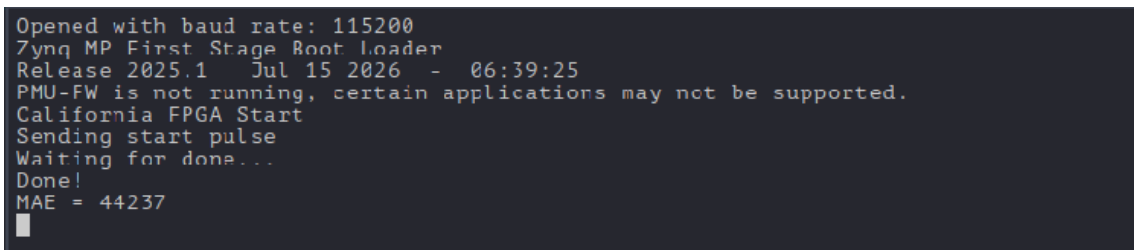
6.2 Hardware Results in VIVADO with or without BRAM



```
restart
INFO: [Wavedata 42-604] Simulation restarted
run all
WARNING: Too many words specified in data file Xtest_160_2.mem
Final MAE = 44237
$finish called at time : 1985165 ns : File "C:/INTERNSHIP/Regression_TSETL
```

Figure 5: MAE from Hardware Implementation

6.3 Hardware Results in VITIS



```
Opened with baud rate: 115200
Zynq MP First Stage Boot Loader
Release 2025.1 Jul 15 2026 - 06:39:25
PMU-FW is not running, certain applications may not be supported.
California FPGA Start
Sending start pulse
Waiting for done...
Done!
MAE = 44237
```

Figure 6: MAE in VITIS platform

6.4 Performance Comparison

To avoid floating-point arithmetic in the hardware implementation, all target values and predictions were scaled by a factor of 10^5 . Consequently, the hardware computes the Mean Absolute Error (MAE) in its scaled integer representation. The software implementation computes the MAE in floating-point format. After accounting for the scaling factor, both implementations produce identical results, thereby validating the correctness of the hardware design. The hardware implementation achieved an MAE of 0.44237, while the software implementation achieved an MAE of 0.44697. The small difference is attributed to fixed-point arithmetic, integer quantization, and truncation effects in the hardware implementation.

7 Conclusion

This internship focused on the hardware implementation of the Regression Tsetlin Machine (RTM) using the AMD Xilinx ZCU104 FPGA platform. The project began with the software implementation of the RTM on the California Housing dataset, where the trained model generated the clause, weight, test feature, and target memory files required for hardware execution.

The complete inference pipeline was implemented in Verilog HDL using Xilinx Vivado, with BRAM used to store the model and test data. The hardware was integrated with the Processing System (PS) through AXI interfaces and deployed using Xilinx Vitis. During development, BRAM latency and controller synchronization issues were identified and resolved, resulting in hardware predictions that matched the software implementation and achieved the same Mean Absolute Error (MAE).

This project demonstrated the feasibility of implementing the Regression Tsetlin Machine on FPGA hardware while providing practical experience in Verilog HDL, FPGA design using Vivado and Vitis, BRAM-based architectures, hardware verification, and hardware–software co-design. The knowledge gained forms a strong foundation for future work in FPGA-based machine learning accelerators.